OPERATING SYSTEMS · FALL 2026 · PROJECT 2 · PROMPT

Variable-Size Cache Manager

Implement pool allocation and cache eviction in the provided `cachelab` starter.

Build a cache of variable-size objects backed by a fixed pool of memory. The program reads a repeatable trace of PUT and GET requests. When a PUT cannot find one contiguous free block large enough for its object, the cache chooses an existing object to evict, frees its block, and tries the allocation again. The payloads are simulated: the pool reserves bytes but stores no file data.

The main work is maintaining an **embedded block list** through allocation, splitting, freeing, and coalescing, then using it with FIFO or clock eviction. Read the separately posted tutorial *From Linked Lists to Embedded Free Lists* before working on `pool.c`.

Build and run

From the extracted `cachelab` directory:

```
make
./cachelab 1024 test_tr/basic.trace 1 -a first -e fifo
```

Before you write any code: The scaffold builds, parses the entire trace, initializes the pool and cache index, starts client threads, and protects each request with the provided locks. On this sample trace, the first PUT reaches the unfinished `handle_put` and exits with `TODO cache.c: implement handle_put` on stderr. It prints no request events or final STATS line. A successful `make` alone is not a working cache.

Usage: `./cachelab <pool_size> <trace_file> <num_clients> -a first|best -e fifo|clock`.

`pool_size` is the **total** number of bytes, including block headers. Use 1–8 clients; use one client first for deterministic, trace-order behavior. Both policy flags are required. The provided driver parses the trace, launches and joins workers, protects each complete request with mutexes, and prints final statistics. Do not add lock calls to the request handlers or the pool: the worker already holds `cache_lock` and `output_lock` while they run.

A trace has one request per line, for example:

```
PUT logo.png 100
GET logo.png
GET missing.png
```

Keys use up to 64 letters, digits, `.`, `_`, or `-`. PUT sizes are positive decimal bytes. Empty lines and lines beginning with `#` are ignored. A PUT reserves payload space under a key; a GET reports HIT if that key is currently stored and MISS otherwise. A miss neither fetches nor allocates an object.

Pool model

The pool begins with a single free block. Every block has a header **inside the pool** followed by payload capacity. The headers are linked in address order. The list includes both free and allocated blocks, unlike the separate insertion-order list of cache entries used for FIFO and clock. In particular, the pool's `next` link is not the cache entry's `policy_next` link.

`block_header_t.size` counts payload capacity, **not** its own header bytes. Search for one free block with capacity at least the request: first-fit takes the first sufficient block; best-fit takes the smallest sufficient block (lowest address on a tie). If a selected block is split, the allocated block is at its front and the remainder gets a new header inside the pool.

The new header must be aligned. Its location is `payload_from_header(block) + round_up(requested, HEADER_ALIGNMENT)`. Let `physical` be that rounded distance. Only split if `physical <= block->size` and `block->size - physical >= MIN_BLOCK_SIZE`, where `MIN_BLOCK_SIZE` is one header plus at least one payload byte. Otherwise the allocated block keeps the extra capacity. For example, with a 232-byte free payload, a 100-byte request rounded to 104 leaves 128 bytes **before** the new header; on a system with a 24-byte header, the new free payload is 104 bytes. Compare 128, not 104, against the minimum *total* block size of 25.

On free, mark the block free, merge with a free successor, then find its predecessor by walking from the list head and merge if that predecessor is free. Each merge recovers the removed header's bytes and emits one COALESCE event. Adjacent free blocks should never remain after `pool_free`. Total free bytes are not enough to satisfy a PUT unless at least one contiguous free block is large enough.

Cache behavior

- Every PUT first reports a PUT event. If its requested size exceeds `pool_size - HEADER_SIZE`, report REJECT and leave all existing entries untouched, including an entry with the same key.
- A valid PUT for a key already present is an UPDATE: remove its old entry from the index and policy list, decrease requested occupancy, and free its old block before placing the replacement. This is **not** policy eviction.
- Try the allocator. On failure choose a FIFO or clock victim, remove and free it using the provided `evict_one` helper, then try allocation again. Continue until one block fits. A non-oversized PUT can eventually succeed after enough evictions and coalescing.
- On success fill the stable `entry_storage` in the current PUT request, insert the new entry in the provided index and policy list, set its reference bit to 1, increase occupancy by its **requested** size, and report ALLOC. Entry metadata lives outside the pool.
- GET uses the provided index. A hit increments hits and sets the entry's reference bit; a miss increments misses. Neither changes pool blocks or insertion order.

FIFO selects the oldest active insertion; a GET does not reorder the list. Clock starts at its persistent hand. For a referenced candidate, clear its bit and advance; for an unreferenced candidate, advance the hand and **return the victim without unlinking it**. The provided `policy_append` initializes the hand for the first entry. The provided `policy_remove` repairs it when removing the entry currently under the hand. `evict_one` performs the actual victim removal after your selection, so do not remove or double-advance the victim in `evict_clock_select`.

Output includes PUT, UPDATE, REJECT, ALLOC, EVICT, COALESCE, HIT, MISS, and a final STATS line. Request and client IDs identify the causing request. The final occupancy is the sum of **requested payload sizes** still cached over the configured total pool size; it excludes headers, padding, and absorbed capacity. With multiple clients, requests can execute in a different order from the trace even though each request and all its output form one protected group. Interpret their events in output order.

What to complete

Only `pool.c` and `cache.c` contain student TODOs. The provided types, parsing, index/list operations, worker synchronization, event formatting, and build rules are kept intact.

1. In `pool.c`, implement `first_fit`, `best_fit`, `split_if_possible`, `pool_try_alloc`, and the free/coalescing body of `pool_free`. The provided pool setup and pointer helpers are available to your code.
2. In `cache.c`, implement `evict_fifo_select`, `evict_clock_select`, `handle_get`, and `handle_put`. Keep the provided `output_event` and `evict_one` code. The latter performs the index/list/occupancy/free transaction for a policy victim; an UPDATE must take a different path.
3. Keep the pool, active cache entries, and policy list consistent after every request. For the embedded list, check that each block ends where the next header begins, all headers are aligned, and the blocks cover the pool without gaps or overlap.

Check your progress

```
make pool-check  # pool-only: splitting, coalescing, contiguity, fit choices
make check       # pool checks plus trace-based cache checks
make clean       # remove generated binary/object files and pool-check binary
```

The pool check is useful after `pool.c` works, before completing `cache.c`. The full check runs `basic.trace` and `frag.trace` under all four policy pairs; targeted runs of other traces check that FIFO and clock choose different victims in `clock_test.trace` and that first-fit and best-fit place objects differently in `placement.trace`. The multi-client checks validate a consistent observed execution rather than demanding trace order. These are public checkpoints, not an exhaustive test suite; test additional split thresholds, updates, fragmentation layouts, and clock-hand states yourself.

Submit the completed `pool.c` and `cache.c` against the provided header, driver, index, parser, and Makefile.

Project 2 materials

The **Project 2 Overview**, **Project 2 Prompt**, and **Pool Tutorial: From Linked Lists to Embedded Free Lists** are all available in **Modules in Canvas**.